



MODUL PERKULIAHAN

SISTEM KENDALI CERDAS

Kode Mata kuliah W132500026

Modul 6

Perancangan Kontrol melalui

Abstrak

Controller digital bekerja dalam siklus deterministik: membaca sensor, memfilter, menghitung error, menjalankan algoritma, menerapkan limit keselamatan, mengirim perintah, dan mencatat data. Timing dan saturasi sama pentingnya dengan persamaan kontrol.

Disusun Oleh :

Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 3.2 — Merancang alur kontrol digital dari sampling hingga perintah actuator



Modul

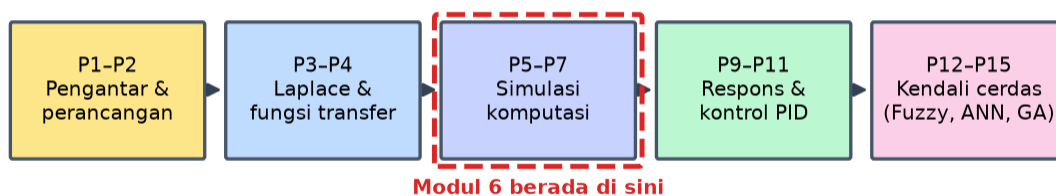
Interaktif:

<https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Sistem-Kendali-Cerdas/Modul/Modul-6.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk membaca seluruh materi tanpa perlu NIM dan PIN. Untuk mengerjakan Tugas dan Forum, masuk sebagai Mahasiswa memakai NIM dan PIN.

PENDAHULUAN

Controller digital bekerja dalam siklus deterministik: membaca sensor, memfilter, menghitung error, menjalankan algoritma, menerapkan limit keselamatan, mengirim perintah, dan mencatat data. Timing dan saturasi sama pentingnya dengan persamaan kontrol.

Alur Mata Kuliah Sistem Kendali Cerdas



Gambar 1. Posisi Pertemuan 6 (Perancangan Kontrol melalui Komputer) dalam alur mata kuliah Sistem Kendali Cerdas.

Modul ini disusun berlapis: pembahasan konsep, penurunan rumus yang dipakai, satu contoh terselesaikan, catatan salah kaprah yang sering terjadi, daftar periksa, lalu tugas terstruktur berupa sepuluh soal pilihan ganda dan lima belas soal komputasi. Versi interaktifnya menilai jawaban secara otomatis di server, sedangkan berkas ini dipakai untuk membaca dan mengerjakan secara luring.

Capaian Pembelajaran (Sub-CPMK)

- Sub-CPMK 3.2 — Merancang alur kontrol digital dari sampling hingga perintah actuator

MENGAPA CONTROLLER DIGITAL BERBEDA DARI RANCANGAN KONTINU

Controller yang berjalan di komputer tidak mengamati sinyal secara terus-menerus. Ia mengambil sampel dari keluaran pada selang tetap, menghitung, lalu menahan keluarannya sampai sampel berikutnya. Tiga kegiatan itu memasukkan sifat yang tidak ada

pada rancangan kontinu, dan mengabaikannya adalah sumber kegagalan yang paling sering terjadi pada penerapan pertama.

Penahanan orde nol membuat sinyal kendali berbentuk tangga. Terhadap plant, tangga tersebut setara dengan tundaan rata-rata sebesar setengah waktu sampling. Tundaan mengurangi margin fase, dan margin fase yang menipis berarti sistem lebih dekat ke ambang ketidakstabilan meskipun penguatannya tidak diubah sama sekali.

Pemilihan waktu sampling karena itu bukan urusan kenyamanan perangkat, melainkan bagian dari perancangan kontrol. Aturan praktis yang lazim menuntut sekitar dua puluh sampai empat puluh sampel sepanjang waktu naik sistem tertutup. Terlalu jarang membuat sistem tidak stabil; terlalu rapat memboroskan sumber daya dan memperkuat derau pada aksi turunan.

Selain waktu sampling, aritmetika terbatas ikut berperan. Bilangan pecahan terbatas menimbulkan pembulatan yang dapat menumpuk pada aksi integral, dan pada perangkat kecil dengan bilangan bulat berskala, pemilihan skala menentukan apakah controller masih berperilaku seperti rancangannya.

penahanan orde nol ~ tundaan $T/2$ | 20 sampai 40 sampel per waktu naik

Diskretisasi: Menerjemahkan Rancangan ke Kode

Controller yang dirancang di domain kontinu harus diterjemahkan menjadi persamaan beda sebelum dapat dijalankan. Beberapa cara tersedia, dan pilihan di antaranya memengaruhi ketepatan serta kestabilan hasil terjemahan.

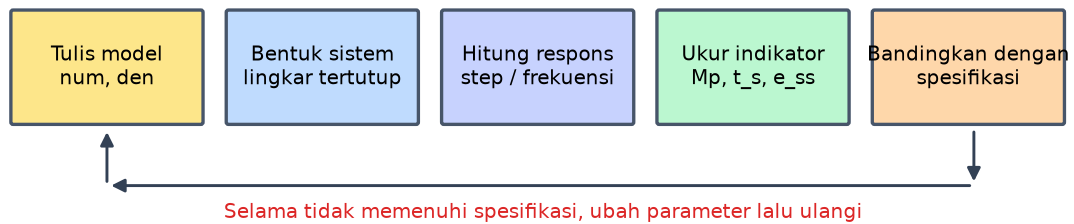
Metode beda mundur sederhana dan selalu memetakan sistem stabil menjadi stabil, namun ketepatannya menurun pada frekuensi tinggi. Metode trapesium, yang juga dikenal sebagai transformasi bilinear, memberi ketepatan lebih baik dan tetap menjaga kestabilan, dengan konsekuensi terjadinya pergeseran frekuensi yang perlu dikoreksi bila titik frekuensi tertentu harus dipertahankan.

Untuk aksi integral, terjemahan yang lazim menambahkan hasil kali error dengan waktu sampling pada akumulator. Untuk aksi turunan, terjemahan yang lazim membagi selisih dua sampel dengan waktu sampling, dan bentuk mentah ini hampir selalu tidak dapat dipakai langsung karena memperkuat derau secara berlebihan.

Alternatif yang lebih baik adalah merancang langsung di domain diskret. Model plant diubah lebih dahulu menjadi bentuk diskret yang memperhitungkan penahanan orde nol, lalu controller dirancang atas model itu. Cara ini menghindari kekeliruan akibat terjemahan dan memperlihatkan pengaruh waktu sampling sejak awal perancangan.

integral: $I \pm K_i e T$ | turunan: $D = K_d (e - e_{\text{lalu}}) / T$, wajib difilter

Alur Kerja Perancangan Berbantuan Komputer



Gambar 2. Alur kerja baku perancangan kontrol memakai perkakas komputasi.

Gambar 2 memperlihatkan bahwa nilai perkakas komputasi terletak pada mudahnya pengulangan: satu putaran yang dahulu memakan waktu sehari-hari kini selesai dalam hitungan detik.

Anti-Windup dan Perpindahan Mode

Ketika actuator mencapai batasnya, keluaran controller tidak lagi berpengaruh pada plant. Aksi integral yang tetap menumpuk selama keadaan itu akan membesar tanpa guna, dan ketika error akhirnya berbalik tanda, akumulator yang telanjur besar harus dikosongkan lebih dahulu sebelum keluaran turun. Akibatnya keluaran melewati setpoint jauh dan lama.

Penanggulangan paling sederhana adalah menghentikan penumpukan ketika keluaran sedang mentok dan error masih mendorong ke arah yang sama. Cara yang lebih halus mengumpanbalikkan selisih antara keluaran yang diminta dan keluaran yang benar-benar terjadi ke akumulator, sehingga akumulator menyesuaikan diri secara mulus alih-alih dibekukan.

Persoalan serupa muncul pada perpindahan antara mode manual dan otomatis. Bila akumulator tidak disiapkan, perpindahan ke otomatis menimbulkan lompatan keluaran yang mengejutkan. Perpindahan tanpa lompatan dicapai dengan menyiapkan akumulator agar keluaran controller pada saat perpindahan sama persis dengan keluaran manual terakhir.

Struktur pengaman ini bukan tambahan opsional. Pada sistem nyata, actuator mentok adalah kejadian biasa, bukan pengecualian, dan perpindahan mode terjadi setiap kali operator mengambil alih. Controller yang tidak menanganinya akan berperilaku baik di simulasi dan mengecewakan di lapangan.

anti-windup: $I \pm K_i e T$ hanya bila tidak mentok, atau $I \pm K_t (u_{\text{nyata}} - u_{\text{minta}})$

Waktu Nyata, Penjadwalan, dan Keandalan

Controller digital harus menyelesaikan perhitungannya sebelum sampel berikutnya tiba. Yang menentukan bukan waktu hitung rata-rata melainkan waktu hitung terburuk, karena satu keterlambatan saja dapat menggeser perilaku sistem. Karena itu tugas kontrol biasanya diberi prioritas tertinggi dan dijauhkan dari kegiatan yang waktunya tidak dapat diperkirakan.

Ketidakteraturan selang sampling sama merusakannya dengan tundaan. Selang yang berubah-ubah membuat aksi integral dan turunan salah menghitung, karena keduanya bergantung langsung pada waktu sampling. Bila selang tidak dapat dijamin tetap, waktu sesungguhnya harus diukur dan dipakai pada perhitungan alih-alih memakai nilai nominal.

Keandalan menuntut penanganan kejadian tidak normal secara eksplisit: sensor yang tidak memberi data, nilai di luar rentang wajar, dan komunikasi terputus. Controller harus memiliki perilaku yang ditetapkan untuk setiap keadaan tersebut, umumnya berupa penahanan keluaran terakhir yang aman disertai pemberitahuan, bukan melanjutkan perhitungan atas data yang tidak sah.

Watchdog melengkapi pengaman dengan memaksa sistem ke keadaan aman bila putaran kendali berhenti berjalan. Perangkat ini bekerja di luar jalur perhitungan utama, tepat karena kegagalan jalur utama itulah yang harus diantisipasi.

waktu hitung terburuk $< T$ | jitter kecil, atau ukur dt sesungguhnya

Verifikasi Kode Kontrol Sebelum Menyentuh Mesin

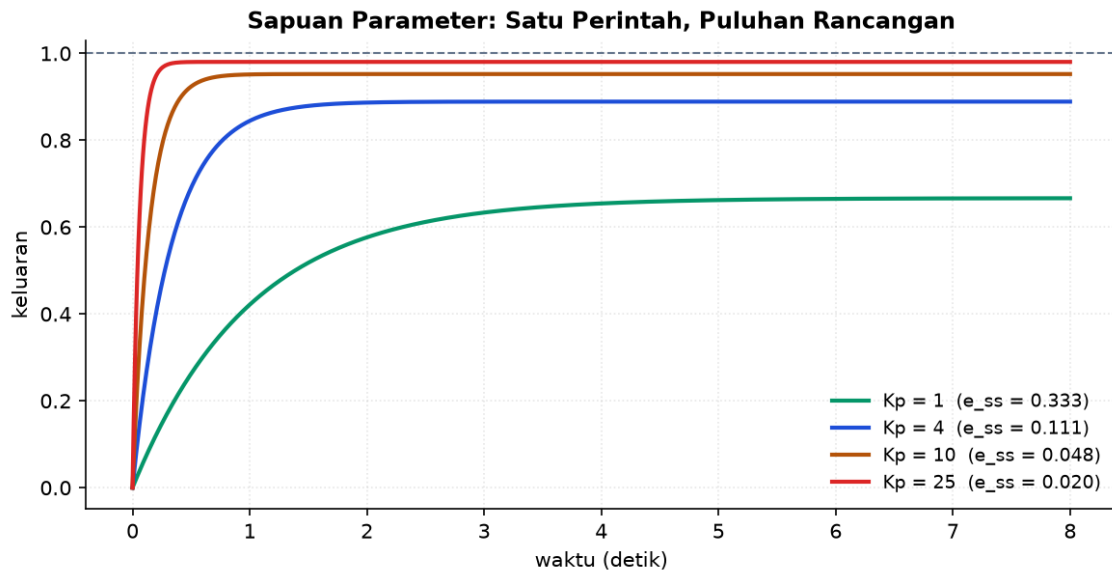
Kode kontrol perlu diuji dengan cara yang sama ketatnya dengan perangkat lunak lain. Fungsi perhitungan controller sebaiknya dipisahkan dari kode yang berurusan dengan perangkat, sehingga dapat diuji sendiri dengan masukan yang ditentukan dan keluaran yang dapat diperiksa.

Pengujian yang paling berharga justru pada keadaan tepi: error nol, error sangat besar, keluaran mentok di kedua arah, waktu sampling berubah, dan perpindahan mode. Keadaan inilah yang jarang muncul di simulasi normal namun sering muncul di lapangan.

Menjalankan kode controller yang sesungguhnya terhadap model plant memberi keyakinan yang jauh lebih besar daripada menjalankan model controller. Cara ini menangkap kekeliruan penerjemahan, kesalahan satuan, dan masalah pembulatan yang tidak akan pernah muncul bila controller ikut disimulasikan sebagai rumus.

Setiap parameter yang dapat diubah operator perlu dibatasi rentangnya di dalam kode. Batas itu melindungi sistem dari nilai yang keliru dimasukkan, dan biayanya hanya beberapa baris dibandingkan kerusakan yang mungkin ditimbulkan.

pisahkan hitung dari perangkat keras | uji keadaan tepi, bukan hanya keadaan normal



Gambar 3. Sapuan gain proporsional pada plant acuan beserta error tunak yang menyertainya.

Gambar 3 menunjukkan cara kerja yang paling banyak dipakai di depan komputer: bukan menghitung satu rancangan, melainkan menyapu rentang parameter lalu memilih dari gambarnya.

Aliasing: Komponen yang Menyamar Setelah Di-sampling

Sampling tidak hanya memperlambat pengamatan; ia juga dapat mengubah komponen berfrekuensi tinggi menjadi komponen berfrekuensi rendah palsu. Gejala inilah yang disebut aliasing, dan begitu terjadi, tidak ada perangkat lunak yang dapat memulihkannya.

Batas yang menentukan adalah setengah frekuensi sampling. Komponen di bawah batas itu terekam apa adanya; komponen di atasnya muncul kembali sebagai frekuensi lain yang lebih rendah dan tidak dapat dibedakan dari sinyal asli. Karena tidak dapat dibedakan, ia akan diperlakukan controller sebagai gangguan sungguhan dan ditanggapi.

Pencegahannya harus dilakukan sebelum sampling, yaitu dengan filter analog yang meredam komponen di atas batas tersebut. Filter digital sesudahnya tidak menolong sama sekali, karena kerusakan sudah terjadi saat sinyal diubah menjadi sampel.

Pada praktiknya, banyak proses lambat yang derau frekuensi tingginya kecil sehingga persoalan ini jarang muncul. Namun pada sistem gerak dengan getaran mekanis, atau pada pengukuran arus dan tegangan, aliasing menjadi penyebab yang nyata dan mudah terlewat karena gejalanya menyerupai gangguan proses biasa.

batas = setengah frekuensi sampling | filter harus analog, sebelum sampling

Aritmetika Terbatas dan Pemilihan Skala

Perhitungan di perangkat sasaran tidak pernah persis. Bilangan pecahan memiliki jumlah angka berarti yang terbatas, dan pada perangkat kecil perhitungan sering dilakukan dengan bilangan bulat berskala demi kecepatan.

Akibat yang paling sering muncul berkaitan dengan aksi integral. Bila tambahan per langkah sampling jauh lebih kecil daripada resolusi bilangan yang dipakai, tambahan itu hilang akibat pembulatan dan akumulator tidak pernah bertambah. Gejalanya khas: error tunak yang tidak pernah hilang meskipun aksi integral tampak aktif.

Pemilihan skala menjadi penentu pada aritmetika bilangan bulat. Skala yang terlalu kecil membuat resolusi kasar sehingga pembulatan merusak; skala yang terlalu besar membuat perhitungan meluap pada nilai ekstrem. Keduanya menghasilkan perilaku yang sulit dilacak karena hanya muncul pada keadaan tertentu.

Cara memeriksanya sederhana: jalankan controller dengan error yang sangat kecil dan amati apakah akumulator benar-benar bertambah, lalu jalankan dengan error terbesar yang mungkin dan amati apakah ada nilai yang meluap. Kedua uji itu murah dan menangkap sebagian besar persoalan aritmetika.

uji akumulator pada error terkecil dan terbesar yang mungkin

Menyusun Program Loop Kendali

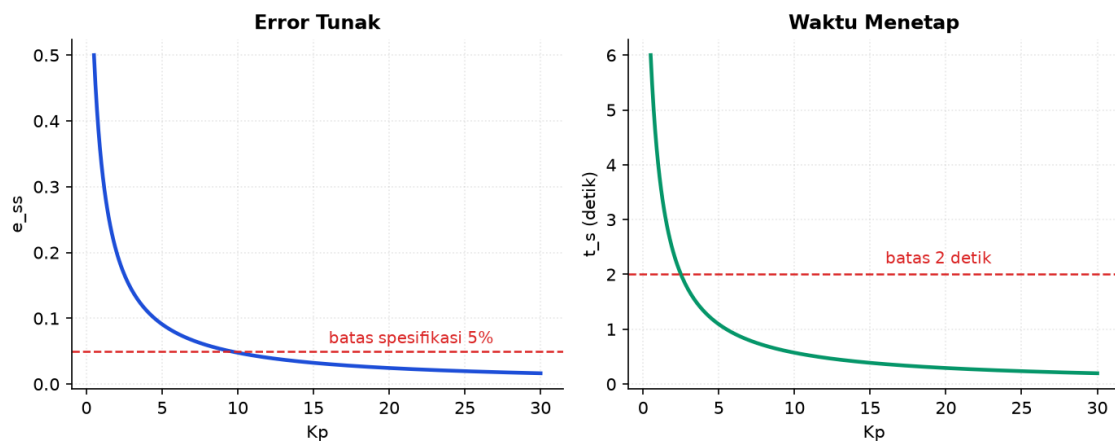
Struktur program yang tertib membuat perilaku controller mudah diperiksa dan mudah diubah. Urutan yang lazim dalam satu langkah sampling: baca sensor, periksa keabsahan data, hitung error, hitung keluaran controller, batasi keluaran, kirim ke actuator, lalu perbarui keadaan internal.

Menempatkan pembatasan keluaran sebelum pengiriman dan sebelum pembaruan akumulator bukan urutan sembarang. Justru urutan itulah yang memungkinkan anti-windup bekerja, karena akumulator dapat disesuaikan berdasarkan selisih antara keluaran yang diminta dan yang benar-benar dikirim.

Pemeriksaan keabsahan data perlu dilakukan sebelum perhitungan, bukan sesudah. Nilai sensor di luar rentang wajar atau yang tidak berubah sama sekali selama beberapa langkah sampling menandakan kegagalan, dan controller harus memiliki perilaku yang ditetapkan untuk keadaan itu alih-alih menghitung atas data yang tidak sah.

Memisahkan fungsi perhitungan dari kode yang berurusan dengan perangkat membuat pengujian jauh lebih mudah. Fungsi perhitungan dapat dijalankan dengan masukan yang ditentukan dan keluarannya diperiksa, tanpa memerlukan perangkat sasaran sama sekali.

baca -> periksa -> hitung -> batasi -> kirim -> perbarui akumulator



Gambar 4. Dua indikator kinerja sebagai fungsi gain, beserta garis batas spesifikasinya.

Gambar 4 mengubah pertanyaan rancangan menjadi pembacaan gambar: rentang gain yang memenuhi seluruh spesifikasi adalah irisan daerah yang berada di sisi aman kedua garis batas.

Menelusuri Masalah Controller Digital di Lapangan

Ketika controller digital berperilaku aneh, penelusuran yang tertib jauh lebih cepat daripada mengubah parameter secara coba-coba. Urutan yang membantu bergerak dari yang paling mudah dipastikan menuju yang paling sulit.

Pemeriksaan pertama adalah keteraturan waktu sampling. Merekam selang antar-sampling selama beberapa menit menampakkan apakah penjadwalnya tertib. Selang yang berubah-ubah menjelaskan banyak gejala sekaligus, karena aksi integral dan turunan sama-sama bergantung padanya.

Pemeriksaan kedua adalah sinyal kendali. Keluaran yang lama bertahan di batas menunjuk ke kejenuhan dan penumpukan integral; keluaran yang bergetar rapat menunjuk ke derau yang diperkuat aksi turunan. Kedua gejala itu terlihat jelas pada grafik sinyal kendali dan hampir tidak terlihat pada grafik keluaran proses.

Pemeriksaan ketiga adalah perbandingan terhadap simulasi memakai kode yang sama. Bila perilaku di perangkat menyimpang jauh dari simulasi yang memakai kode identik, selisihnya berasal dari hal yang tidak ada di simulasi: waktu sampling, aritmetika, derau, atau perangkat. Menyempitkan penyebab dengan cara ini jauh lebih cepat daripada menebak.

telusuri: keteraturan sampling -> sinyal kendali -> banding dengan simulasi

MENURUNKAN BENTUK DISKRET PID UNTUK DIPROGRAM

Penurunan berikut mengubah PID kontinu menjadi persamaan beda yang langsung dapat ditulis menjadi kode.

1. PID kontinu

$$u(t) = K_p * e + K_i * \text{integral}(e \, dt) + K_d * de/dt$$

Tiga aksi bekerja atas error yang sama.

2. Dekati integral

$$\text{integral}(e \, dt) \sim \text{sum}(e_k * T)$$

Luas di bawah kurva didekati sebagai jumlah persegi selebar T.

3. Dekati turunan

$$de/dt \sim (e_k - e_{(k-1)})/T$$

Selisih dua sampel berurutan dibagi selang waktunya.

4. Bentuk posisi

$$u_k = K_p * e_k + K_i * T * \text{sum}(e) + K_d * (e_k - e_{(k-1)})/T$$

Menghitung keluaran secara mutlak; memerlukan penanganan windup tersendiri.

5. Bentuk kenaikan

$$du = K_p * (e_k - e_{(k-1)}) + K_i * T * e_k + K_d * (e_k - 2 * e_{(k-1)} + e_{(k-2)})/T$$

Diperoleh dengan mengurangkan $u_{(k-1)}$ dari u_k .

6. Keluaran akhir

$$u_k = u_{(k-1)} + du$$

Bentuk kenaikan menangani windup secara alami karena pembatasan pada u_k langsung menghentikan penumpukan.

Bentuk kenaikan lebih disukai pada penerapan industri karena perpindahan mode menjadi mulus dengan sendirinya dan pembatasan keluaran sekaligus berperan sebagai anti-windup. Aksi turunan pada kedua bentuk tetap memerlukan filter agar derau sensor tidak diperkuat.

CONTOH TERHITUNG: MEMILIH WAKTU SAMPLING

Diketahui: Sistem tertutup dirancang memiliki waktu naik sekitar 0,4 detik · Sensor menghasilkan derau yang cukup berarti pada frekuensi tinggi

1. Terapkan aturan sampling

$$T = t_r/30 = 0,4/30$$

Diambil tiga puluh sampel sepanjang waktu naik, di tengah rentang anjuran.

2. Hitung waktu sampling

$$T = 0,0133 \text{ s}$$

Dibulatkan ke nilai yang mudah dijadwalkan, yaitu 0,01 detik atau 100 hertz.

3. Perkirakan tundaan tambahan

$$T/2 = 0,005 \text{ s}$$

Penahanan orde nol setara tundaan setengah waktu sampling.

4. Ubah menjadi susut fase

$$\text{fase} = \omega_c * T/2, \text{ dengan } \omega_c \sim 2/t_r = 5 \text{ rad/s}$$

Diperoleh 0,025 radian atau sekitar 1,4 derajat.

5. Nilai dampaknya

$$\text{margin fase berkurang sekitar } 1,4 \text{ derajat}$$

Masih kecil terhadap margin fase rancangan yang umumnya 45 sampai 60 derajat.

Jawaban. Waktu sampling 0,01 detik memadai: cukup rapat sehingga susut fase akibat penahanan hanya sekitar 1,4 derajat, dan tidak terlalu rapat sehingga aksi turunan tidak memperkuat derau secara berlebihan. Filter turunan dengan frekuensi potong sekitar sepuluh kali bandwidth tertutup dapat ditambahkan tanpa mengganggu kinerja.

SALAH KAPRAH YANG SERING TERJADI

- Memakai penguatan rancangan kontinu tanpa memeriksa waktu sampling — Tundaan akibat penahanan mengurangi margin fase. Sistem yang stabil di atas kertas dapat berosilasi saat diprogram.
- Menerapkan aksi turunan tanpa filter — Selisih dua sampel memperkuat derau sensor. Actuator akan bergetar terus-menerus dan cepat aus.
- Melupakan anti-windup — Actuator mentok adalah kejadian biasa. Tanpa penanganan, keluaran melewati setpoint jauh melebihi perkiraan simulasi.
- Membiarkan selang sampling tidak teratur — Aksi integral dan turunan bergantung langsung pada T. Selang yang berubah membuat keduanya salah hitung.

- Menguji hanya keadaan normal — Kekeliruan paling mahal muncul pada keadaan tepi: error besar, keluaran mentok, dan perpindahan mode.

DAFTAR PERIKSA SEBELUM LANJUT

- ☐ Waktu sampling dipilih dari waktu naik target, bukan dari kemudahan perangkat
- ☐ Susut fase akibat penahanan orde nol diperhitungkan
- ☐ Aksi turunan dilengkapi filter dan pembatasan penguatan
- ☐ Anti-windup diterapkan dan diuji pada keadaan mentok
- ☐ Perpindahan manual ke otomatis diuji bebas lompatan
- ☐ Waktu hitung terburuk diukur dan lebih kecil daripada waktu sampling
- ☐ Perilaku saat sensor gagal atau data tidak sah ditetapkan eksplisit
- ☐ Kode controller yang sesungguhnya diuji terhadap model plant

TUGAS

Tugas dikerjakan pada halaman modul interaktif agar penilaiannya otomatis. Bobotnya 50 poin: sepuluh soal pilihan ganda @1 poin, sepuluh soal komputasi tingkat dasar-menengah @2 poin, dan lima soal komputasi tingkat lanjut @4 poin. Soal komputasi dikerjakan dengan Python; yang dinilai adalah nilai yang dicetak, bukan cara penulisan programnya.

A. Pilihan Ganda (10 soal @1 poin)

1. Penahanan orde nol pada keluaran controller digital, terhadap plant, setara dengan...
 - A. Penguatan tambahan sebesar dua kali
 - B. Tundaan rata-rata sebesar setengah waktu sampling
 - C. Filter lolos tinggi
 - D. Zero tambahan di sebelah kanan sumbu imajiner
2. Aturan praktis pemilihan waktu sampling menuntut sekitar...
 - A. 2 sampai 5 sampel sepanjang waktu naik
 - B. 20 sampai 40 sampel sepanjang waktu naik
 - C. 100 sampel per detik berapa pun sistemnya
 - D. Satu sampel per konstanta waktu plant

3. Aksi turunan diskret yang dihitung sebagai selisih dua sampel dibagi waktu sampling, tanpa filter, hampir selalu menimbulkan...
- A. Error tunak yang membesar
 - B. Penguatan derau sensor sehingga actuator bergetar
 - C. Overshoot yang hilang sama sekali
 - D. Kegagalan konvergensi solver
4. Bentuk kenaikan PID lebih disukai pada penerapan industri terutama karena...
- A. Perhitungannya lebih singkat
 - B. Tidak memerlukan aksi turunan
 - C. Perpindahan mode menjadi mulus dan pembatasan keluaran sekaligus berperan sebagai anti-windup
 - D. Tidak memerlukan waktu sampling tetap
5. Selang sampel yang tidak teratur merusak perhitungan terutama pada aksi...
- A. Proporsional saja
 - B. Integral dan turunan, karena keduanya bergantung langsung pada waktu sampling
 - C. Turunan saja
 - D. Tidak ada; PID tidak peka terhadap keteraturan selang
6. Yang menentukan apakah controller sanggup berjalan waktu nyata adalah...
- A. Waktu hitung rata-rata
 - B. Waktu hitung terburuk
 - C. Jumlah baris kode
 - D. Kecepatan jam prosesor saja
7. Mekanisme watchdog berguna karena...
- A. Mempercepat perhitungan controller
 - B. Menggantikan kebutuhan anti-windup
 - C. Bekerja di luar jalur perhitungan utama dan memaksa sistem ke keadaan aman bila putaran kendali berhenti
 - D. Menghaluskan sinyal kendali

8. Perpindahan dari mode manual ke otomatis tanpa lompatan dicapai dengan...
- Menolkan seluruh gain saat perpindahan
 - Menyiapkan akumulator integral agar keluaran controller sama dengan keluaran manual terakhir
 - Menonaktifkan sensor sesaat
 - Memperbesar waktu sampling sementara
9. Merancang langsung di domain diskret lebih unggul daripada menerjemahkan rancangan kontinu karena...
- Menghindari kekeliruan penerjemahan dan menampakkan pengaruh waktu sampling sejak awal perancangan
 - Selalu menghasilkan gain yang lebih besar
 - Tidak memerlukan model plant
 - Menghilangkan kebutuhan anti-windup
10. Menjalankan kode controller yang sesungguhnya terhadap model plant memberi keyakinan lebih besar daripada mensimulasikan controller sebagai rumus karena menangkap...
- Gangguan lingkungan di lapangan
 - Keausan mekanis
 - Kesalahan pemilihan sensor
 - Kekeliruan penerjemahan, kesalahan satuan, dan masalah pembulatan

B. Komputasi Dasar-Menengah (10 soal @2 poin)

Parameter acuan: Plant orde satu dengan $K = 3$ dan $\tau = 8$ s dikendalikan PID digital. Rancangan menargetkan waktu naik loop tertutup 6 detik, dan waktu sampling dipilih dengan aturan 30 sampel sepanjang waktu naik. Gain loop rancangan $L = 5$. Sampel error terakhir: $e_k = 2,0$; $e_{(k-1)} = 2,3$; $e_{(k-2)} = 2,5$.

Tabel 1. Parameter acuan yang dipakai seluruh soal komputasi modul ini.

Parameter	Nilai
K / τ	$3 / 8$ s
t_r target	6 s
K_p, K_i, K_d	2 ; 0,5 ; 1,5
L	5

C1. Tentukan waktu sampling rancangan, dalam detik. Cetak: T.

– PETUNJUK: Langkah: 1) ambil waktu naik target 6 detik. 2) pakai aturan 30 sampel sepanjang waktu naik. 3) $T = t_r/30$.

C2. Hitung tundaan setara akibat penahanan orde nol, dalam detik. Cetak: tundaan.

– PETUNJUK: Langkah: 1) peroleh T. 2) penahanan orde nol setara tundaan setengah waktu sampling. 3) hitung $T/2$.

C3. Hitung susut fase akibat penahanan orde nol pada frekuensi kerja, dalam derajat. Cetak: susut fase (derajat).

– PETUNJUK: Langkah: 1) perkirakan frekuensi kerja $\omega_c = 2/t_r$. 2) peroleh tundaan $T/2$. 3) susut fase = $\omega_c \times (T/2)$ radian. 4) ubah ke derajat dengan mengalikan $180/\pi$.

C4. Bila waktu sampling dinaikkan menjadi 0,3 detik, hitung berapa sampel yang terjadi sepanjang waktu naik. Cetak: jumlah sampel.

– PETUNJUK: Langkah: 1) ambil waktu naik target. 2) ambil waktu sampling baru. 3) bagi keduanya. 4) bandingkan dengan anjuran 20 sampai 40 sampel.

C5. Hitung tambahan akumulator integral pada satu langkah sampling untuk error saat ini. Cetak: tambahan integral.

– PETUNJUK: Langkah: 1) peroleh T. 2) ambil K_i dan e_k dari parameter acuan. 3) tambahan = $K_i \times e_k \times T$.

C6. Hitung aksi turunan diskret pada langkah sampling saat ini. Cetak: aksi turunan.

– PETUNJUK: Langkah: 1) peroleh T. 2) hitung selisih e_k dikurangi $e_{(k-1)}$. 3) bagi selisih itu dengan T. 4) kalikan K_d .

C7. Hitung frekuensi Nyquist sistem sampel ini, dalam hertz. Cetak: f_{Nyquist} (Hz).

– PETUNJUK: Langkah: 1) peroleh T. 2) frekuensi sampling = $1/T$. 3) Nyquist adalah setengahnya, yaitu $1/(2T)$.

C8. Hitung rasio waktu sampling terhadap konstanta waktu plant. Cetak: T/τ .

– PETUNJUK: Langkah: 1) peroleh T. 2) ambil τ plant. 3) bagi T dengan τ .

C9. Hitung konstanta waktu loop tertutup rancangan, dalam detik. Cetak: τ_{cl} .

– PETUNJUK: Langkah: 1) ambil τ plant. 2) ambil gain loop L. 3) hitung $1+L$. 4) $\tau_{cl} = \tau/(1+L)$.

C10. Hitung berapa sampel yang terjadi sepanjang waktu menetap pita dua persen. Cetak: jumlah sampel.

– PETUNJUK: Langkah: 1) hitung τ_{cl} . 2) $t_s = 4 \times \tau_{cl}$. 3) peroleh T. 4) bagi t_s dengan T.

C. Komputasi Lanjut (5 soal @4 poin)

C11 (Lanjut). Hitung perubahan keluaran du pada satu langkah sampling memakai PID bentuk kenaikan. Cetak: du.

– PETUNJUK: Langkah: 1) tulis $du = K_p(e_k - e_{(k-1)}) + K_i T e_k + K_d(e_k - 2e_{(k-1)} + e_{(k-2)})/T$. 2) peroleh T. 3) hitung suku proporsional. 4) hitung suku integral. 5) hitung pembilang suku turunan $e_k - 2e_{(k-1)} + e_{(k-2)}$. 6) selesaikan suku turunan lalu jumlahkan ketiganya.

C12 (Lanjut). Filter aksi turunan dipilih berfrekuensi potong sepuluh kali bandwidth loop tertutup. Hitung rasio konstanta waktu filter terhadap waktu sampling. Cetak: τ_f/T .

– PETUNJUK: Langkah: 1) hitung τ_{cl} dari τ dan L. 2) bandwidth tertutup kira-kira $1/\tau_{cl}$. 3) kalikan sepuluh untuk memperoleh frekuensi potong. 4) konstanta waktu filter = $1/\text{frekuensi potong}$. 5) peroleh T. 6) bagi konstanta waktu filter dengan T.

C13 (Lanjut). Perhitungan controller memakan 45 milidetik pada kondisi terburuk dan penjadwal menimbulkan ketidakaturan 15 milidetik. Hitung margin waktu yang tersisa dalam satu langkah sampling, dalam detik. Cetak: margin (detik).

– PETUNJUK: Langkah: 1) peroleh T. 2) ubah 45 ms menjadi detik. 3) ubah 15 ms menjadi detik. 4) jumlahkan keduanya sebagai beban terburuk. 5) kurangkan dari T. 6) nilai apakah margin itu memadai.

C14 (Lanjut). Controller meminta keluaran 12 satuan sementara actuator hanya mampu 10 satuan. Dengan skema anti-windup berumpan balik ber- $K_t = 1/T$, hitung koreksi akumulator pada satu langkah sampling. Cetak: koreksi akumulator.

– PETUNJUK: Langkah: 1) peroleh T. 2) hitung $K_t = 1/T$. 3) hitung selisih keluaran nyata dikurangi keluaran diminta. 4) tulis koreksi = $K_t \times \text{selisih} \times T$. 5) hitung nilainya. 6) tentukan arah koreksinya, menambah atau mengurangi akumulator.

C15 (Lanjut). Operator memindahkan mode dari manual ke otomatis saat keluaran manual 6,5 satuan dan error saat itu 2,0. Hitung nilai akumulator integral yang harus disiapkan agar perpindahan tidak melompat. Cetak: akumulator.

– PETUNJUK: Langkah: 1) tulis keluaran PID sebagai jumlah suku P, I, dan D. 2) pada saat perpindahan, aksi turunan diambil nol karena error belum berubah. 3) ambil K_p dan error. 4) hitung suku proporsional. 5) syaratkan keluaran total sama dengan keluaran manual terakhir. 6) selesaikan nilai akumulator.

DAFTAR PUSTAKA

Fuller, S., dkk. (2021). The Python Control Systems Library (python-control). IEEE Conference on Decision and Control, 4875-4881.

Virtanen, P., dkk. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17(3), 261-272.

Harris, C. R., dkk. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.

Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2019). Feedback control of dynamic systems (8th ed.). Pearson.